UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

MATERIA: Fundamentos de Inteligencia Artificial

DOCENTE: Mtro. Javier García Lara

ALUMNA: Denise Estrada Franco

TEMA: 2.1 Problemas de búsqueda
2.2 Espacios de búsqueda

SEMANA 1: 12-18 octubre

GRUPO: 171

CARRERA: Tecnologías de la Información e Innovación Digital

MATRÍCULA: 24305019

FECHA DE ENTREGA: 18 de octubre de 2026

1. PLANTEAMIENTO DEL PROBLEMA

En esta sección se modelan dos problemas reales como problemas de búsqueda en espacios de estados, enfoque en el que se definen estados, acciones y objetivos para explorar posibles soluciones (Russell & Norvig, 2020).

Cada problema se formaliza definiendo estados, acciones, coste y meta; se representa como un grafo dirigido y ponderado; se implementa un generador de estados en Python junto con la construcción del grafo mediante NetworkX; se entrega el diagrama correspondiente y se realiza el análisis de complejidad. Esta formulación sigue los conceptos generales de los problemas de búsqueda (Russell & Norvig, 2020).

Problema 1 – Ruta de entrega urbana (courier / last-mile delivery)

Un repartidor de comida a domicilio debe desplazarse desde un restaurante hasta la vivienda del cliente utilizando una red de calles de un barrio. Algunas calles son de un solo sentido y cada tramo tiene un coste real expresado en minutos de viaje (según distancia y tráfico estimado). El objetivo es encontrar una secuencia de intersecciones que conecte el origen con el destino.

Problema 2 – Navegación de un robot de almacén

Un robot móvil autónomo debe desplazarse desde su estación de carga hasta el estante donde se encuentra un paquete. El almacén se modela como una cuadrícula de 4x4 celdas con obstáculos fijos (estantes y pilares). El robot solo puede moverse en las cuatro direcciones cardinales hacia celdas libres. El objetivo es alcanzar la celda del paquete minimizando el número de movimientos.

2. FORMULACIÓN FORMAL (ESTADO, ACCIONES, COSTE, META)

Problema 1 – Red de calles

ELEMENTO	DEFINICIÓN
ESTADO	Intersección / nodo de la red (etiqueta: A, B, C …)
ESTADO INICIAL	Restaurante (nodo A)
ACCIONES	Transitar por una calle dirigida hacia un nodo adyacente
MODELO DE TRANSICIÓN	Resultados (s, a) = s’ donde existe arco dirigido (s → s’)
COSTE DE ACCIÓN	Peso del arco (minutos de viaje)
META	Cualquier estado s tal que s == Cliente (nodo G)

COSTE DE CAMINO	Suma de los pesos de los arcos recorridos

Tabla 1. Formulación Formal. Problema 1. Elaboración propia

Problema 2 – Robot de almacén

ELEMENTO	DEFINICIÓN
ESTADO	Par de coordenadas (x, y) con $0 \leq x, y \leq 3$ y celda libre
ESTADO INICIAL	(0, 0) – estación de carga
ACCIONES	Norte, Sur, Este, Oeste (si la celda destino está libre y dentro de límites)
MODELO DE TRANSICIÓN	(x, y) + Δ (x', y') si es válida
COSTE DE ACCIÓN	1 (movimiento unitario)
META	(3, 2) – ubicación del paquete
COSTE DE CAMINO	Número de movimientos realizados

Tabla 2. Formulación Formal. Problema 2. Elaboración propia

3. PROCEDIMIENTO

1. Se definió formalmente cada problema mediante la quíntupla clásica de búsqueda (estado, acciones, transición, coste, prueba de meta).
2. Se implementó un generador de estados (función de sucesores) para cada problema.
3. Se construyó un grafo dirigido y ponderado a partir del generador de estados utilizando la biblioteca NetworkX.
4. Se generó el diagrama visual de cada grafo con matplotlib, diferenciando nodo inicial (verde), nodo meta (rojo) y nodos intermedios.
1. Se realizó el análisis de complejidad teórica de la construcción del grafo y de las búsquedas clásicas, tomando como referencia la teoría de algoritmos de búsqueda (Bohm, 2021; Russell & Norvig, 2020).
2.

4. SOLUCIÓN – DIAGRAMAS DE LOS GRAFOS

Grafo del Problema 1 – Red de Calles

El grafo contiene 7 nodos (intersecciones) y 11 arcos dirigidos ponderados. El nodo A representa el restaurante (inicio) y el nodo G representa la

ubicación del cliente (meta). Los pesos indican el tiempo estimado de recorrido en minutos.

Búsqueda de la ruta óptima mediante Dijkstra

Se implementó el algoritmo de Dijkstra para encontrar la ruta de menor coste desde el restaurante (nodo A) hasta el cliente (nodo G). El algoritmo considera los pesos de los arcos, que representan el tiempo estimado de recorrido en minutos. El resultado obtenido fue la ruta A → C → D → G, con un coste total de 7 minutos. En el diagrama, esta ruta se resalta en color naranja para distinguirla de las demás conexiones de la red.

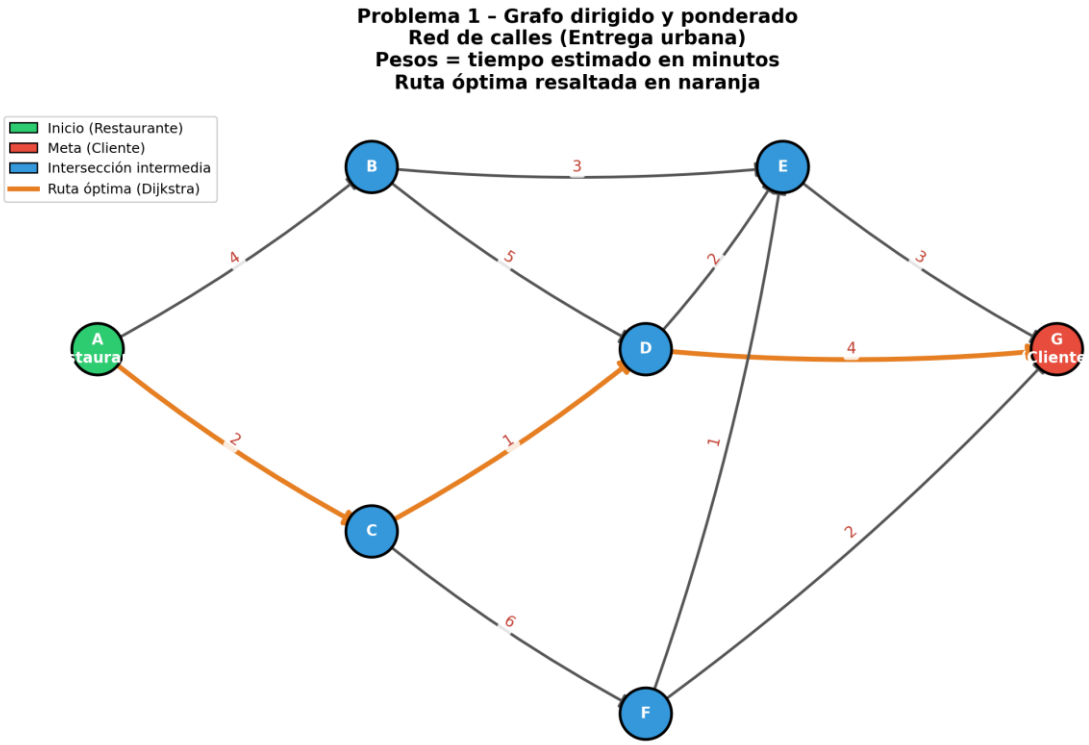


Figura 1. Grafo dirigido y ponderado de la red de calles (Problema 1).

Grafo del Problema 2 – Robot de almacén

El espacio de estados se construyó expandiendo el generador de sucesores desde el estado inicial. Se obtuvieron 13 estados alcanzables y 32 arcos. Cada arco tiene

coste unitario = 1. Los obstáculos del mapa reducen el espacio de 16 celdas posibles a 13 estados válidos.

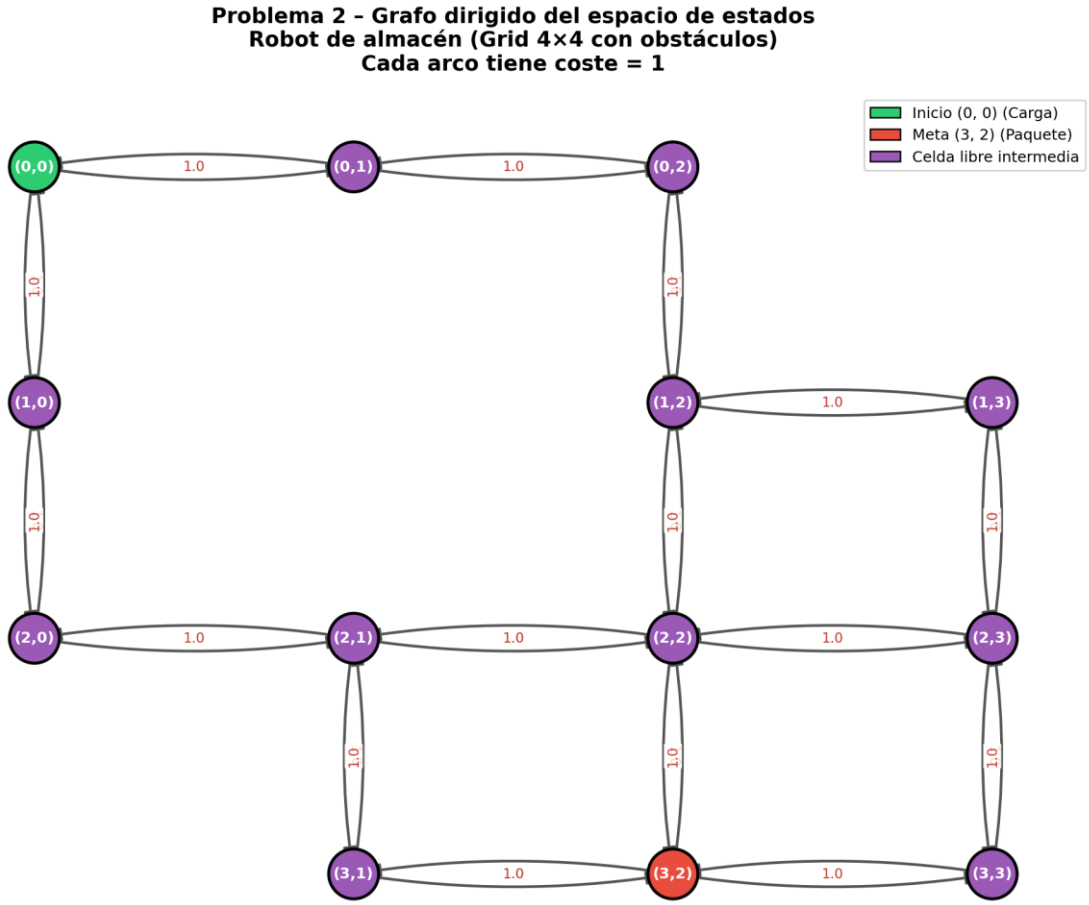


Figura 2. Grafo dirigido del espacio de estados del robot de almacén (Problema 2)

5. CÓDIGO PYTHON COMENTADO

A continuación, se presentan los fragmentos más relevantes del código. El archivo completo **2problemasS2.py** se encuentra en el repositorio GitHub.

5.1 Generador de estados – Problema 1

```python
def generador_estados_calles(G: nx.DiGraph, estado: str) -> List[Tuple[str, float]]:
    """
    Generador de estados (sucesores) para el problema de calles.
    Devuelve lista de (estado_sucesor, coste_accion).
    """
    sucesores = []
    for vecino in G.successors(estado):
        coste = G[estado][vecino]["weight"]
        sucesores.append((vecino, coste))
    return sucesores
```

Ilustración 1. Generador de estados. Problema 1. Elaboración propia

5.2 Generador de estados – Problema 2

```python
def generador_estados_robot(estado: Tuple[int, int]) -> List[Tuple[Tuple[int, int], float]]:
    """
    Generador de estados para el robot.
    Acciones: Norte, Sur, Este, Oeste. Coste unitario = 1.
    """
    x, y = estado
    movimientos = [(-1, 0), (1, 0), (0, 1), (0, -1)]
    sucesores = []
    for dx, dy in movimientos:
        nx_, ny_ = x + dx, y + dy
        if es_valido(nx_, ny_):
            sucesores.append(((nx_, ny_), 1.0))
    return sucesores
```

Ilustración 2. Generador de Estados. Problema 2. Elaboración propia

5.3 Construcción del grafo del robot a partir del generador

```python
def crear_grafo_robot() -> nx.DiGraph:
    """
    Construye el grafo dirigido completo del espacio de estados del robot
    a partir del generador de estados.
    """
    G = nx.DiGraph()
    visitados: Set[Tuple[int, int]] = set()
    cola = [INICIO]
    visitados.add(INICIO)

    while cola:
        actual = cola.pop(0)
        for sucesor, coste in generador_estados_robot(actual):
            G.add_edge(actual, sucesor, weight=coste)
            if sucesor not in visitados:
                visitados.add(sucesor)
                cola.append(sucesor)
    return G
```

Ilustración 3. Construcción del grafo del robot. Problema 2. Elaboración propia

6. ANÁLISIS DE COMPLEJIDAD

Problema 1 (Grafo de Calles):

- n= 7 nodos, m = 11 arcos.
- Generación de sucesores: O (grado del nodo) ≈ O (1) en promedio.
- Construcción del grafo: O (n+m)
- Espacio de estados: O (n) – finito y pequeño
- Búsqueda no informada (BFS/DFS): O(n +m).
- Dijkstra / A*: O((n + m) log n) con montículo binario.

Problema 2 (Grid del Robot)

- Grid N ×N= 4 × 4 → máximo 16 celdas; con obstáculos ≈ 13 estados alcanzables.
- Factor de ramificación b ≈ 2-3
- Generación de sucesores: O (1) (4 direcciones constantes)
- Construcción del grafo por expansión: O (número de estados) = O (N^2)
- Búsqueda no informada: O(bd) en el peor caso teórico; en la práctica O (estados alcanzables) porque se evita revisitar nodos
- Con A* + heurística admisible (distancia Manhattan) la eficiencia mejor significativamente.

7. CONCLUSIONES

Ambos problemas reales se modelaron como problemas de búsqueda mediante la quíntupla formada por estado, acciones, modelo de transición, coste y prueba de meta. Esta formalización permitió definir claramente las condiciones iniciales, los movimientos posibles y los objetivos de cada problema. La formulación se basa en los principios generales de representación de problemas de búsqueda (Russell & Norvig, 2020).

La representación mediante grafos dirigidos y ponderados permitió describir la red de calles y el espacio de estados del robot de almacén. Asimismo, el uso de NetworkX facilitó la construcción y visualización de los grafos, mientras que los generadores de estados permitieron identificar las transiciones válidas en cada problema.La biblioteca NetworkX documenta estructuras y operaciones para la construcción y visualización de grafos (NetworkX Developers, 2024).

En el problema de la red de calles se implementó el algoritmo de Dijkstra para determinar la ruta de menor coste desde el restaurante hasta el cliente. El resultado fue la ruta A → C → D → G, con un tiempo estimado de recorrido de 7 minutos. Esto demuestra la utilidad de considerar los pesos de las conexiones al resolver problemas de búsqueda en los que se busca minimizar el coste total.

Por otra parte, el problema del robot de almacén permitió representar los movimientos posibles dentro de una cuadrícula con obstáculos, considerando únicamente las celdas accesibles y asignando un coste unitario a cada movimiento. De esta manera, se ejemplificó cómo un problema de navegación puede representarse mediante un espacio de estados finito.

Finalmente, el análisis de complejidad permitió identificar que la construcción y exploración de estos grafos es manejable debido a su tamaño reducido. En problemas de mayor escala, sería necesario evaluar algoritmos de búsqueda más eficientes y, cuando corresponda, utilizar heurísticas que permitan orientar la exploración hacia el objetivo

8. REFERENCIAS

Referencias

Bohm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science Publishers.

NetworkX Developers. (2024). *NetworkX documentation*. https://networkx.org/

Python Software Foundation. (2024). *Python language reference*. https://www.python.org/

Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Wirsansky, E. (2020). *Hands-on genetic algorithms with Python*. Packt.

9. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICAL

Herramienta: Chat GPT Open IA

Prompt resumido: Modela dos problemas reales como problemas de búsqueda, representar como grafo dirigido/ponderado, programar generador de estados y grafo con NetworkX.

Uso: Apoyo en la formalización de los problemas y generación del código base

Verificación: El código fue ejecutado localmente, los grafos se generaron correctamente, los diagramas se revisaron visualmente y el análisis de complejidad se contrastó con la teoría de Russell & Norvig y Bohm.